Aula 05 — Simulação na prática: rodando um experimento de fila

Avaliação de Desempenho (PC010029)

Prof. Dr. Helvecio Bezerra Leal Neto

Universidade Federal do Oeste do Pará

8 de setembro de 2026 ·

Cronograma

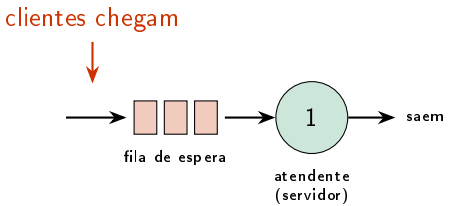
1. Resolução analítica do problema
2. Implementação do modelo em Python
3. Definição das taxas: λ, μ e ρ

   Intervalo (10 min)
4. Incorporação de estocasticidade
5. Análise estatística e replicações
6. Aplicação prática e síntese

Meta

Ler, alterar e rodar um simulador de fila.

O sistema de hoje: uma fila e um atendente

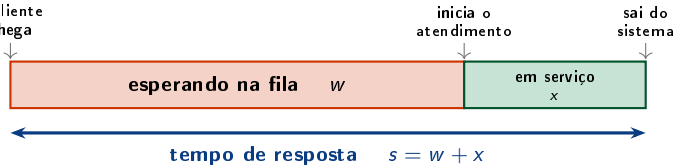


As regras

- Um atendente só.
- Quem chega e encontra ele ocupado **espera na fila**.
- Atende-se na ordem de chegada (FIFO).

É a fila do banco, do RU, da impressora compartilhada e da CPU atendendo processos.

O que queremos medir



	Nome	Onde o cliente está
w	tempo de espera	**na fila**, aguardando
x	tempo de atendimento	**no centro de serviço**
s	tempo de resposta	**no sistema**: fila + serviço

Tempo médio de resposta

$$E[s] = E[w] + E[x]$$

Além dos tempos: a **utilização** U, fração do tempo com o atendente ocupado.

O exemplo

Três clientes. Para cada um sabemos **quando chega** e **quanto tempo precisa** do atendente.

c_1	c_2	c_3
chega em $t = 0$	chega em $t = 2$	chega em $t = 6$
precisa de 3 s	precisa de 2 s	precisa de 2 s

As perguntas

Quanto cada um **espera**? Quando cada um **sai**? Que fração do tempo o atendente **trabalhou**?

Vamos responder montando uma tabela, cliente por cliente.

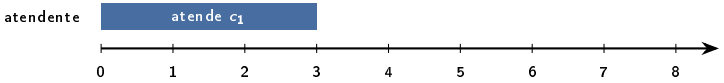
Montando a tabela: cliente 1

	chega	inicia	sai	espera
c_1	0	**0**	**3**	**0**
c_2	2	?	?	?
c_3	6	?	?	?

Raciocínio

Em $t = 0$ o atendente está **livre**: c_1 inicia na hora.

inicia $= 0$ sai $= 0 + 3 = 3$ espera $= 0 - 0 = 0$



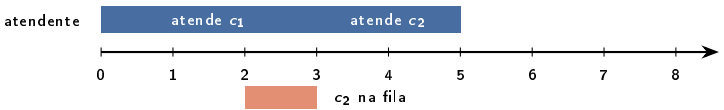
Montando a tabela: cliente 2

	chega	inicia	sai	espera
c_1	0	0	3	0
c_2	2	**3**	**5**	**1**
c_3	6	?	?	?

Raciocínio

Em $t = 2$ o atendente está **ocupado** com c_1 até $t = 3$: c_2 **espera na fila**.

inicia $= 3$ sai $= 3 + 2 = 5$ espera $= 3 - 2 = 1$



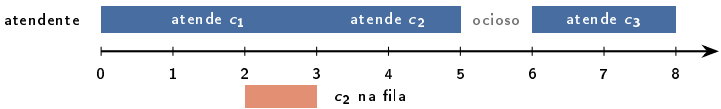
Montando a tabela: cliente 3

	chega	inicia	sai	espera
c_1	0	0	3	0
c_2	2	3	5	1
c_3	6	**6**	**8**	**0**

Raciocínio

c_2 saiu em 5. Em $t = 6$ o atendente está **livre** de novo: c_3 inicia na hora.

inicia $= 6$ sai $= 6 + 2 = 8$ espera $= 6 - 6 = 0$



A regra que usamos três vezes

Quando um cliente chega

- Atendente **livre**: inicia agora.
- Atendente **ocupado**: inicia quando o anterior sair.

Depois disso, só contas

$\text{sai} = \text{inicia} + \text{serviço}$

$\text{espera} = \text{inicia} - \text{chega}$

Guarde essa regra: é exatamente ela que o programa vai executar daqui a pouco, só que 500 vezes.

Da tabela às respostas: tempo de resposta

	chega	sai	espera w	resposta s
c_1	0	3	0	$3-0=\mathbf{3}$
c_2	2	5	1	$5-2=\mathbf{3}$
c_3	6	8	0	$8-6=\mathbf{2}$

Resposta s = sai − chega

Tudo o que o cliente viveu no sistema: a espera **mais** o serviço, $s = w + x$.

Tempos médios: $E[\cdot]$

$E[s]$ **(tempo médio de resposta):** $\dfrac{3+3+2}{3} = \dfrac{8}{3} = 2{,}67\,\text{s}$

$E[w]$ **(tempo médio de espera):** $\dfrac{0+1+0}{3} = 0{,}33\,\text{s}$

Da tabela às respostas: utilização



Tempo ocupado: quanto trabalhou

Somando os serviços: $3 + 2 + 2 = 7$ s.

Tempo total: quanto durou

Acabou quando c_3 saiu: 8 s.

Utilização (U) = ocupado ÷ total

$U = 7/8 = 0{,}875 = 87{,}5\%$. Ficou parado $8 - 7 = 1$ s, entre $t = 5$ e $t = 6$.

Você acabou de simular

Laço de eventos

Cada chegada e cada saída foi um **evento**: seis ao todo. A cada um, olhamos o estado (atendente livre?) e decidimos.

O relógio saltou

$0 \to 2 \to 3 \to 5 \to 6 \to 8$: um salto por evento. Ninguém contou o tempo de um em um.

Na aula anterior vimos uma tabela tinha uma linha por evento; a de hoje, uma por cliente. O conteúdo é o mesmo. **Simulação a eventos discretos** é isto: lista de eventos, relógio que salta, regra de decisão.

A pergunta de hoje

Três clientes cabem no papel. E quinhentos?

À mão

Três linhas levaram cinco minutos. Quinhentas levariam a tarde inteira, com erro garantido.

Simulador (Código)

As mesmas linhas saem em milésimos de segundo. E trocar um parâmetro custa um caractere.

O plano: escrever o programa, conferir que ele devolve **2,67 s e 87,5%**, e só então soltá-lo em cenários que não conseguimos calcular.

O SimPy já traz pronto o que você fez à mão

O que é o SimPy?

Framework de simulação de eventos discretos em Python. Modela entidades ativas (clientes, veículos) como processos geradores e gerencia recursos com capacidade limitada.

Na tabela	No SimPy	O que você escreve
O relógio ($t = 0, 2, 3 \ldots$)	`env`	`env.now`
A lista de eventos	interna à biblioteca	nada
"sai = inicia + serviço x"	`timeout`	`yield env.timeout(x)`
"atendente livre?" e a fila	`Resource`	`with servidor.request()`

A troca

Você deixa de fazer a tabela e passa a descrever o **comportamento de um cliente**.

Antes de tudo: como rodar

```
$ pip install simpy
$ python3 fila_v1.py
```

- `pip` instala bibliotecas; roda uma vez só.
- `python3 arquivo.py` executa o arquivo.

Onde

No terminal do laboratório, no seu notebook ou no Google Colab (célula com `!pip install simpy`).

Passo 1: `import` traz código pronto

```
import simpy

env = simpy.Environment()
```

- Uma **biblioteca** é código que outra pessoa já escreveu e testou.
- `import` deixa esse código disponível no seu arquivo.

Notação de ponto

O ponto acessa recursos da biblioteca importada. Por exemplo, `simpy.Environment` acessa o `Environment` dentro de `simpy`.

Analogia

Você não fabrica a calculadora para somar. Pega a pronta.

Passo 2: estrutura e uso de um objeto

```
env = simpy.Environment()
```

Cria uma instância do ambiente e armazena na variável `env`.

```
env.now   # Atributo: estado atual
env.run() # Metodo: executa a acao
```

O que é um objeto?

Estrutura que agrupa **dados** (atributos) e as **ações** (métodos) que operam sobre eles.

Regra de sintaxe

- **Sem parênteses**: acessa um atributo (dado).
- **Com parênteses**: executa um método (ação).

Passo 3: função é uma receita com nome

```
def dobro(x):
    return 2 * x

dobro(4)     # devolve 8
```

- `def` define; o nome vem em seguida.
- Entre parênteses, o que ela recebe.
- `return` é o que ela devolve.

Por que funções

Escrevemos **uma vez** o que um cliente faz e chamamos 500 vezes.

Indentação

As linhas deslocadas para a direita **pertencem** ao `def`. Em Python o recuo é obrigatório.

Passo 4: `yield` é uma função que sabe pausar

```
def contar():
    print("comecei")
    yield 1
    print("voltei")
    yield 2
g = contar()  # so preparou a receita
next(g)       # roda ate o yield: comecei
next(g)       # continua: voltei
```

A diferença

Função comum roda até o fim e morre. Com `yield`: **para, guarda onde estava** e espera alguém mandar continuar.

`contar()` só prepara a receita; `next(g)` executa até o próximo `yield`. No SimPy quem chama o `next` é o `env.run()`: você nunca escreve `next`.

Peça 1: env é o relógio da simulação

```
import simpy

env = simpy.Environment()
# servidor: recurso compartilhado (
    atendente)
# chegadas: processo gerador de clientes
env.process(chegadas(env, servidor))
env.run()
print(env.now)    # 8
```

Ambiente e Tempo

- **Ambiente**: gerencia o relógio e a fila de eventos.
- `env.now`: tempo simulado atual, distinto do relógio real.

Processos e Execução

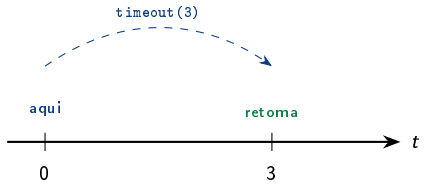
- `env.process()`: registra funções geradoras no escalonador.
- `env.run()`: executa os eventos em ordem cronológica.

Peça 2: `yield env.timeout(t)` é o tempo passando

```
yield env.timeout(3)
```

Modela a passagem do tempo na simulação:

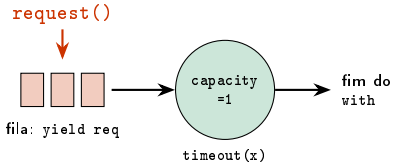
- `env.timeout(3)`: agenda evento com atraso de 3 unidades.
- `yield`: suspende o processo e entrega controle ao `env`.
- Simulação: o `env` avança o tempo e reativa o processo.



É o "sai = inicia + 3" da tabela.

Peça 3: Resource gerencia capacidade limitada e filas

```
servidor = simpy.Resource(env, capacity=1)
with servidor.request() as req:
    yield req                # aguarda na fila
    yield env.timeout(x)     # atende o processo
```



- **Recurso**: entidade com capacidade finita compartilhada entre processos concorrentes.
- `request()`: requisita uso; `yield req` aguarda vaga na fila.
- Bloco `with`: reserva o recurso e garante liberação automática.

As três variáveis que guardam o experimento

```
CHEGADAS = [0, 2, 6]    # quando cada cliente aparece
SERVICOS = [3, 2, 2]    # quanto tempo cada um ocupa o servidor
saidas   = []           # cada linha vira (nome, chegada, w, saida)
```

Lista

`[0, 2, 6]` é uma **lista**: valores em ordem, entre colchetes. `saidas.append(x)` acrescenta `x` ao fim da lista.

Por que MAIÚSCULAS

Convenção de Python para **constantes**: valores que você ajusta no topo e não mexe mais.

Posição 1 de cada lista é o mesmo cliente: c_2 chega em 2 e é servido por 2 (os cartões do slide 5). `saidas` é a nossa tabela.

Duas coisas que o script usa para falar com você

```
nome = "c2"
print(f"{env.now:4.1f}  {nome} chega")    # mostra:   2.0  c2 chega
```

print

`print` = mostre na tela. Todo o log e a tabela dos próximos slides são só `print`s.

f"..."

Texto com valores encaixados: cada {} vira o valor da variável. `:4.1f` = 1 casa decimal em 4 espaços. Só formata; não muda o valor.

O script (1/3): quem chega, e quando

```
def chegadas(env, servidor):
    anterior = 0
    for i, t in enumerate(CHEGADAS):
        yield env.timeout(t - anterior)
        anterior = t
        env.process(cliente(env, f"c{i+1}", servidor, SERVICOS[i]))
```

- `enumerate(CHEGADAS)`: repete o bloco para cada chegada; `i` é a posição (0, 1, 2), `t` o valor (0, 2, 6).
- `t - anterior`: o `timeout` conta **a partir de agora**, só a diferença até a próxima chegada.
- `f"c{i+1}"` monta o nome `c1`, `c2`, `c3`; `SERVICOS[i]` é o serviço da mesma posição.

O script (2/3): tudo o que UM cliente faz

```
def cliente(env, nome, servidor, x):
    chegada = env.now
    print(f"{env.now:4.1f}  {nome} chega")
    with servidor.request() as req:
        yield req
        w = env.now - chegada
        print(f"{env.now:4.1f}  {nome} inicia (esperou {w:.1f} s)")
        yield env.timeout(x)
        print(f"{env.now:4.1f}  {nome} sai")
        saidas.append((nome, chegada, w, env.now))
```

Leia como uma receita, de cima para baixo

Peço o atendente e espero a vez. O `yield req` acorda: ele é meu, e w é "agora menos a chegada". Sou atendido por x e guardo a minha linha da tabela; no fim do `with` o atendente é devolvido.

O script (3/3): a partida

```
env = simpy.Environment()
servidor = simpy.Resource(env, capacity=1)
env.process(chegadas(env, servidor))
env.run()
```

`env.run()`

Roda até a agenda esvaziar: quando c_3 sai, em $t = 8$.

E a tabela?

No fim do arquivo, 8 linhas percorrem `saidas` e montam a tabela do slide 29: são só contas e `prints`.

A saída: a tabela que montamos, impressa linha a linha

```
0.0  c1 chega
0.0  c1 inicia (esperou 0.0 s)
2.0  c2 chega
3.0  c1 sai
3.0  c2 inicia (esperou 1.0 s)
5.0  c2 sai
6.0  c3 chega
6.0  c3 inicia (esperou 0.0 s)
8.0  c3 sai
```

Confira com o slide 7

c_2 chegou em 2 e só iniciou em 3: **esperou 1 s**, o retângulo vermelho.

Mesmos saltos do relógio

$0 \to 2 \to 3 \to 5 \to 6 \to 8$, linha a linha.

As contas que o script imprime no fim

```
cliente chega espera sai resposta
  c1     0.0   0.0   3.0   3.0
  c2     2.0   1.0   5.0   3.0
  c3     6.0   0.0   8.0   2.0

resposta media = (3+3+2)/3 = 2.67 s
utilizacao = ocupado/total = 7/8 = 87.5%
```

As mesmas contas dos slides 10 e 11

resposta $=$ sai $-$ chega

ocupado $= 3 + 2 + 2 = 7$, total $= 8$

No código

```
ocupado = sum(SERVICOS)
total = env.now
```

O momento da verdade

	À mão	SimPy
c_2 esperou	1,0 s	1,0 s
Resposta média $E[s]$	2,67 s	2,67 s
Utilização	87,5%	87,5%

Por que isso importa

O script acertou onde já sabíamos a resposta. **É essa conferência que autoriza** usá-lo onde não sabemos.

Nunca rode um simulador em cenário desconhecido sem antes testá-lo num caso que você consegue conferir.

Tempos fixos não descrevem sistema nenhum

O que temos

Chegadas fixas em 0, 2 e 6 min: determinístico, resultado sempre idêntico.

O que queremos

Média de 1,25 min: estocástico, com clientes próximos ou espaçados.

Para isso, precisamos de dois elementos:

- **Taxa de chegada**: frequência média de eventos por minuto.
- **Sorteio aleatório**: intervalos gerados a partir da taxa.

(Daqui em diante usamos minutos, não segundos; o script é o mesmo.)

As taxas do nosso exemplo

Taxa de chegada (λ)

1. **Total:** 3 chegadas em 8 s.
2. **Tempo médio:** $E[t] = 8/3 \approx 2{,}67$ s.
3. **Taxa:**

$\lambda = 1/E[t] = 3/8 = 0{,}375$ cliente/s.

Taxa de serviço (μ)

1. **Total:** $3 + 2 + 2 = 7$ s (3 serviços).
2. **Tempo médio:** $E[x] = 7/3 \approx 2{,}33$ s.
3. **Taxa:**

$\mu = 1/E[x] = 3/7 \approx 0{,}43$ cliente/s.

Regra de bolso: taxa e tempo médio são inversos. Um a cada 2 s $\Leftrightarrow$ 0,5 por segundo.

ρ: a fração do tempo que o atendente precisa trabalhar

Intensidade de tráfego (Aula 03)

$$\rho = \frac{\lambda}{\mu} = \frac{\text{o que chega}}{\text{o que ele dá conta}}$$

No nosso exemplo

$$\rho = \frac{0{,}375}{0{,}43} = \frac{3/8}{3/7} = \frac{7}{8} = 0{,}875$$

Deu 87,5%, o mesmo U do slide 11. Batem **exatamente** porque λ e μ saíram destes mesmos três clientes.

$\rho \geq 1$: chega mais do que sai e a fila cresce sem parar. Estável só com $\lambda < \mu$.

Intervalo

10 minutos

Ao voltar: de onde o computador tira os números sorteados.

O computador não sorteia: ele calcula

PRNG

Pseudo **R**andom **N**umber **G**enerator: gerador de números **pseudo**aleatórios.

A receita

$$x_{\text{novo}} = (a \cdot x_{\text{velho}} + c) \bmod m$$

mod = o resto da divisão. Vamos usar $a = 5$, $c = 3$ e $m = 16$.

Pseudo = parece sorteado, mas é uma **receita fixa**: de um número sai o próximo.

O `random` do Python faz exatamente isso, com a, c e m gigantescos.

E isso é bom

Com acaso de verdade, ninguém conseguiria repetir o seu experimento, nem você mesmo na semana seguinte.

A receita à mão: comece em 7 e gire a manivela

Começando pela **semente** $x_0 = 7$:

$(5 \cdot 7 + 3) \bmod 16 = 38 \bmod 16 = 6$
$(5 \cdot 6 + 3) \bmod 16 = 33 \bmod 16 = 1$
$(5 \cdot 1 + 3) \bmod 16 = 8 \bmod 16 = 8$
$(5 \cdot 8 + 3) \bmod 16 = 43 \bmod 16 = 11$

Para cair entre 0 e 1: divida por m

$6/16 = 0{,}375$	$1/16 = 0{,}0625$
$8/16 = 0{,}500$	$11/16 = 0{,}6875$

A saída de cada volta é a entrada da próxima. São estes os números que o `random.random()` devolve, e deles saem os tempos da simulação.

A semente: o número por onde a receita começa

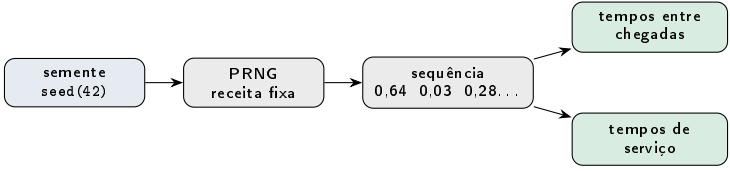
Semente	Sequência que sai			
$x_0 = 7$	6	1	8	11
$x_0 = 3$	2	13	4	7

Mesma receita; só o **ponto de partida** mudou.

Por que isso importa

Mesma semente, mesma sequência, mesmo resultado. É o que torna o seu experimento **repetível** por outra pessoa.

[Johnson & Coutinho, 2011 · Cap. 8 · pág. 127]



Não queremos números entre 0 e 1: queremos tempos

O PRNG entrega 0,64, 0,03, 0,28... Precisamos de **minutos entre uma chegada e a outra**.

Como devem ser esses tempos?

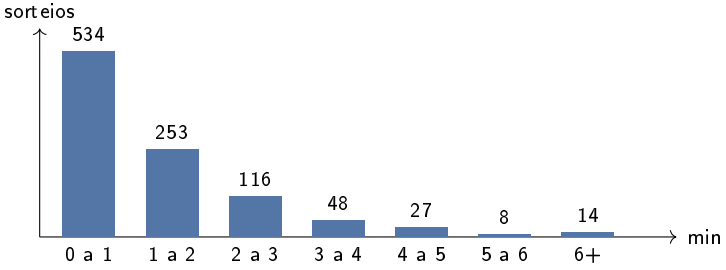
Clientes reais não chegam de relógio na mão. O comum é o intervalo **curto**; o intervalo longo existe, mas é raro.

A distribuição que faz isso

A **exponencial**. Em Python: `random.expovariate`.

Vamos sortear com $\lambda = 0{,}8$ cliente/min, a taxa que usaremos na versão 2: em média, um cliente a cada $1/0{,}8 = 1{,}25$ min.

A cara da exponencial: muitos curtos, poucos longos



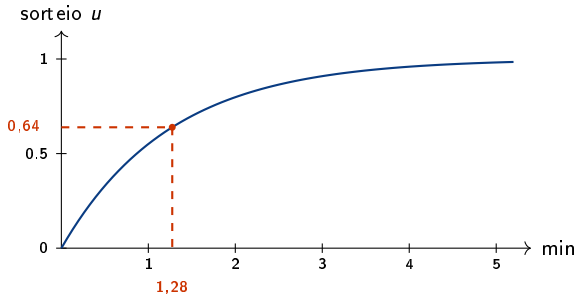
1000 sorteios

Mais da metade abaixo de 1 min; só 14 acima de 6.

A média confere

Média dos 1000: **1,32** min, perto de $1/\lambda = 1{,}25$.

Do sorteio para o tempo



Como ler a curva

A curva diz, para cada tempo, **que fração dos intervalos é menor que ele**.

No nosso sorteio

$u = 0{,}64$: entro pela esquerda na altura 0,64, bato na curva e desço. Leio **1,28 min**.

Sortear u perto de **0** cai na parte baixa da curva e devolve tempo **curto**; perto de **1**, sobe para a cauda e devolve tempo **longo**.

Fatiando o tempo

1. Numa fatia curtinha de tempo Δt, a chance de chegar alguém é $\lambda \Delta t$. Logo a chance de **não** chegar é $1 - \lambda \Delta t$.

2. Para não chegar ninguém em x minutos, corte x em n fatias de tamanho x/n e não chegue em **nenhuma** delas:

$$\left(1 - \frac{\lambda x}{n}\right)^n$$

3. Quanto mais finas as fatias, esse número para de mudar. Com $\lambda x = 1$:

fatias n	10	100	1000	$\rightarrow$
resultado	0,349	0,366	0,368	$e^{-1} = 0{,}368$

O limite tem nome

É o $e^{-\lambda x}$: a fração dos intervalos **maiores** que x.

O que sobra são os **menores**:

$$u = 1 - e^{-\lambda x}$$

Isolando o x

$$1 - u = e^{-\lambda x}$$

$$\ln(1-u) = -\lambda x$$

$$x = \frac{-\ln(1-u)}{\lambda}$$

Com $u = 0{,}639$, $\lambda = 0{,}8$: $x = \dfrac{-\ln(1-0{,}639)}{0{,}8} = \dfrac{-\ln(0{,}361)}{0{,}8} = \dfrac{1{,}020}{0{,}8} = 1{,}28$ min

Cuidado: `expovariate` recebe a taxa, não o tempo médio

Certo

Quero um cliente a cada **1,25 min**.
Taxa: $\lambda = 1/1{,}25 = 0{,}8$.
`random.expovariate(0.8)`

Errado

`random.expovariate(1.25)`
Média: $1/1{,}25 = 0{,}8$ min.
Chegam **mais** clientes do que você queria.

O erro não gera **nenhuma mensagem**: o programa roda e devolve números plausíveis.

Checagem: para um atendimento que dura 4 min em média, o que vai dentro do `expovariate`?

A versão 2: os tempos fixos viram sorteios

```
LAMBDA = 0.8        # por minuto
MU = 1.0            # por minuto
SERVIDORES = 1
N_CLIENTES = 500
SEMENTE = 42
MOSTRAR_LOG = False

esperas = []        # a rodada preenche
servicos = []
fila = []
```

O que mudou (três coisas)

1. `expovariate(lam)` no lugar de `t - anterior`.
2. `expovariate(mu)` no lugar do `s` da lista.
3. `range(n)`: repita `n` vezes (500), sem lista para percorrer.

A versão 2, o cliente (1/2): chega e espera

```
def cliente(env, nome, servidor, mu, log):
    chegada = env.now
    with servidor.request() as req:
        fila.append(len(servidor.queue))
        yield req
        w = env.now - chegada
        esperas.append(w)
```

O que é novo

`servidor.queue` = quem está esperando: anotamos o tamanho da fila a cada chegada. A espera vai para a lista `esperas`, em vez de para o `print`.

A versão 2, o cliente (2/2): é atendido por um tempo sorteado

```
        if log:
            print(f"{env.now:5.2f}  {nome} inicia (esperou {w:.2f})")
        x = random.expovariate(mu)
        servicos.append(x)
        yield env.timeout(x)
```

O que é novo

`if log:` só mostra a linha quando `MOSTRAR_LOG` é `True`. O serviço `x` é **sorteado** com a taxa `mu` e guardado em `servicos`: é daí que sai o tempo ocupado.

A versão 2: as chegadas

```
def chegadas(env, servidor, lam, mu, n, log):
    for i in range(n):                          # repita n vezes
        yield env.timeout(random.expovariate(lam))
        env.process(cliente(env, f"c{i+1}", servidor, mu, log))
```

range(n)

Repita `n` vezes; com `n = 500`, quinhentos clientes. Não há mais lista para percorrer.

Os botões viajam como parâmetros

`lam`, `mu`, `n`, `log` recebem os valores do topo; outro arquivo pode passar outros.

A versão 2: a partida, dentro de rodar

```
def rodar(lam=LAMBDA, mu=MU, servidores=SERVIDORES,
          n=N_CLIENTES, semente=SEMENTE, log=MOSTRAR_LOG):
    random.seed(semente)
    esperas.clear(); servicos.clear(); fila.clear()
    env = simpy.Environment()
    servidor = simpy.Resource(env, capacity=servidores)
    env.process(chegadas(env, servidor, lam, mu, n, log))
    env.run()
    ocupado = sum(servicos)
    total = env.now
```

`lam=LAMBDA`: se ninguém passar outro valor, vale o do topo. `clear()` esvazia as listas. No fim do arquivo, `rodar()` é chamada e as contas são impressas (slide 49).

Rodando a versão 2

No topo de `fila_v2.py`: `N_CLIENTES = 6` e `MOSTRAR_LOG = True`. O `print` do cliente mostra:

```
1.28  c1 inicia (esperou 0.00)
1.60  c2 inicia (esperou 0.29)
2.93  c3 inicia (esperou 1.31)
5.16  c4 inicia (esperou 2.12)
5.19  c5 inicia (esperou 2.04)
5.43  c6 inicia (esperou 1.60)
```

O `:5.2f` só formata: 2 casas decimais em 5 espaços.

Repare

c_1 chegou em 1,28: o primeiro sorteio, o do slide 40.

Já tem fila

c_3 esperou 1,31 min: os dois anteriores chegaram colados.

As contas da versão 2, com 500 clientes

```
rho = lambda/mu = 0.8/1.0 = 0.80
espera media = soma das esperas / n = 2955.6/500 = 5.91 min
ocupado = 548.5 min   total = 658.4 min
utilizacao   = ocupado/total = 548.5/658.4 = 0.833 = 83.3%
fila maxima  = 23 clientes
```

As mesmas contas

`sum(esperas)/500`; `ocupado = sum(servicos)`; `total = env.now`; `max(fila)`.

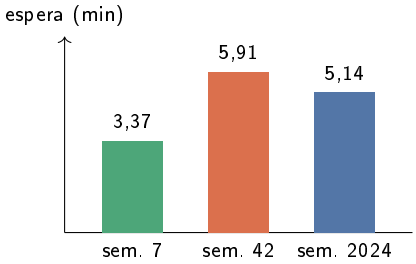
Confere com a previsão?

Previmos $\rho = 0{,}8$; medimos $U = 0{,}833$. A diferença é o acaso de uma rodada só.

Pergunta: podemos anotar 5,91 min e ir para casa?

Nenhuma linha mudou. Só a semente.

Semente	Espera	U	Fila máx.
7	3,37	73,0%	18
42	5,91	83,3%	23
2024	5,14	87,5%	13



75% a mais

$5{,}91 \div 3{,}37 = 1{,}75$. Não há bug: mesmo modelo, outra sequência de sorteios.

Para reproduzir: troque `SEMENTE` no topo de `fila_v2.py` e rode de novo.

Uma rodada é uma amostra, não uma medida

A analogia

Reportar uma rodada é medir o tempo de resposta de um servidor **uma vez só** e publicar aquele número.

Regra da disciplina

Nunca informe um resultado sem dizer **quantas rodadas** e **quais sementes**.

Não confunda com modelo falho

Variação entre sementes é esperada. Resultado errado por carga mal gerada (`expovariate(5)` querendo média 5) é bug. [Johnson & Coutinho, 2011 · Cap. 1 · pág. 22-23]

Teste 1: se λ cresce, a espera tem de piorar

Mantemos $\mu = 1{,}0$, um atendente e a semente 42. Só a taxa de chegada muda.

λ	$\rho = \lambda/\mu$	U medido	Espera medida	Confere?
0,50	0,50	0,507	1,06 min	sim
0,80	0,80	0,833	5,91 min	sim
0,95	0,95	0,940	10,34 min	sim

A previsão se confirmou

O U medido ficou colado no ρ calculado nas três linhas.

Mas não é proporcional

λ quase dobrou e a espera ficou **dez vezes** maior.

Teste 2: com $\rho > 1$, um atendente não dá conta

Agora $\lambda = 1{,}2$ e $\mu = 1{,}0$: chega mais gente do que ele despacha.

m	$\rho = \lambda/(m\mu)$	Previsão	U medido	Espera	Fila máx.
1	1,20	fila explode	0,996	24,80 min	54
2	0,60	fila some	0,644	0,90 min	12

Com $m = 1$ ele não parou

$U = 99{,}6\%$. A espera só não é infinita porque a rodada acabou.

Com $m = 2$, previsão certa

A espera caiu de 24,8 para 0,9 min. No código, é trocar `SERVIDORES`.

É para isso que serve simular: responder "compensa um segundo atendente?" sem contratar ninguém.

Atividade avaliativa: a sua fila, com a sua matrícula

O que fazer

Replicar a simulação da **versão 2** (a fila com tempos sorteados) trocando a semente pelo **seu número de matrícula**.

Por que a matrícula

Cada um recebe uma sequência de sorteios diferente: os seus números não serão os do colega.

Prazo e valor

Entrega no **SIGAA** até **sexta 11/09, 23:59**.

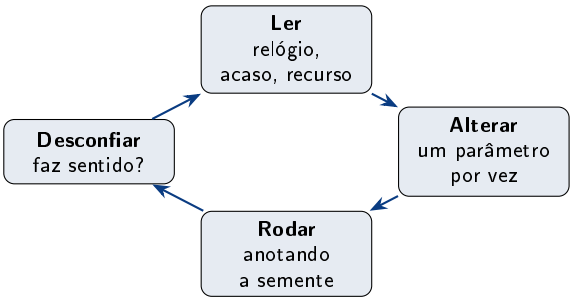
Vale **1,5 ponto**.

O que o seu código precisa conter

1. **Parâmetros no topo**, com nome: λ, μ, atendentes, nº de clientes e a `MATRICULA` como semente.
2. **Rodada base** com $\lambda = 0{,}8$: o log dos 6 primeiros clientes e as métricas com as contas à mostra (ρ, espera, ocupado, total, U, fila máxima).
3. **Teste 1**: rodar com $\lambda = 0{,}5$, 0,8 e 0,95. A espera piorou? O U ficou perto do ρ?
4. **Teste 2**: $\lambda = 1{,}2$ com 1 e com 2 atendentes. O segundo atendente resolveu?
5. **Teste 3**: rodar duas vezes com a mesma semente. Deu igual? Por quê?
6. **Conclusão** em 3 a 5 linhas: o simulador passou nos testes?

Código limpo conta ponto: parâmetros nomeados no topo, sem números soltos no meio, um comentário curto por bloco.

Autonomia cabe em quatro verbos



O que o modelo não é

"Modelar um sistema não significa reproduzi-lo em cada detalhe." Aqui ninguém desiste da fila nem fura fila: escolhemos deixar isso fora, não esquecemos. [Johnson & Coutinho, 2011 · pág. 106-107]

Próxima aula: Aferição, medição e monitoração (Aula 06, 15/09)

- A terceira técnica: medir o sistema real.
- Monitores, contadores e o custo de instrumentar.
- Simulação $\times$ medição: outro par de validação.

Traga

Seu script rodando.

Referências I

- Jain, R. *The Art of Computer Systems Performance Analysis*. John Wiley & Sons, 1991. Cap. 24 e 25.
- Johnson, T. M. S. M. *Avaliação de desempenho de sistemas computacionais*. TrioStudio, 2011. Cap. 1, 2, 6, 8 e 13.
- Allen, A. *Probability, Statistics, and Queueing Theory with Computer Science Applications*. Academic Press, 1978.
- Monks, T. et al. *An introduction to Discrete-Event Simulation using Free and Open Source Software* (intro-open-sim). `https://github.com/pythonhealthdatascience/intro-open-sim`
- Team SimPy. *SimPy: Discrete event simulation for Python*. `https://simpy.readthedocs.io`